

徐锦源个人任务及感想

1. 基本信息

项目	内容
姓名	徐锦源
学号	2024611176
组内角色	models 开发者，负责算法内核
对应模块	app/models/
主要 issue	#3 T2 算法内核

2. 本人承担的任务

我在本项目中主要负责拓扑排序应用软件的算法内核部分。该部分位于 `app/models/`，属于纯逻辑层，不依赖 PySide6，也不直接处理界面渲染。我的任务是按老师任务书中“按 `<a,b>` 输入关系、判断关系图、输出尽可能多 拓扑排序结果、考虑异常情况”的要求落实为可复用的模型接口，供后续事件层、渲染层和 UI 层调用。

具体承担内容包括：

- 实现关系文本解析，支持标准 `<A,B>` 输入，并兼容空白行、全角括号、全角逗号、缺少尖括号的 `A,B` 形式。
- 实现图数据结构，保存节点和边，处理重复边去重，并在自环输入时返回 明确错误。
- 自研 Kahn 拓扑排序相关算法，包括环检测、最长路径分层、全部拓扑序枚举 和 仅计数模式。
- 补充与算法内核对应的测试数据、期望结果、模型测试和性能记录。
- 修复 models 相关 CI 隔离问题，保证纯逻辑层导入时不会加载 Qt。

3. 具体产出

可核实的关联提交：

提交	说明
21eb899	feat: 实现拓扑排序算法内核 (refs #3)
6f3647a	fix: 隔离 models CI 的 Qt 插件加载 (refs #3)

本地和 GitHub 页面中暂无可核实的个人 PR 记录；本说明按当前仓库 git 提交 记录和 issue #3 归档。

主要修改文件如下：

- `app/models/parser.py`：实现输入解析、宽容格式处理、`ParseError` 行号错误报告。
- `app/models/graph.py`：实现不可变图模型、节点/边查询、环检测入口和 `CycleError`。
- `app/models/enumerator.py`：实现 Kahn 分析、拓扑序枚举、计数模式和 卡住节点识别。
- `app/models/__init__.py`：整理算法内核公共 API。
- `app/tests/test_models.py`：补充模型层行为测试，覆盖标准图、宽容格式、环、自环、坏输入和大图计数。
- `app/tests/conftest.py`、`app/tests/test_events.py`：配合标准图期望数量 修正测试口径。

- [.github/workflows/ci.yml](#)：修正 models CI 的 Qt 插件加载隔离问题。
- [deliverables/04-详细设计.md](#)：撰写算法内核详细设计章节。
- [evidence/decisions/2026-09-18-CycleError异常契约.md](#)：记录环图分层 异常契约。
- [evidence/decisions/2026-09-18-标准图拓扑序数量修正.md](#)：记录标准图 拓扑序数量修正结论。
- [evidence/test-data/001-标准五节点.in](#) 至 [evidence/test-data/008-多余括号非法.expected](#)：补充算法测试输入和期望。
- [evidence/benchmarks.csv](#)：追加 30 节点、60 节点稀疏链计数性能记录。

这些产出使算法层形成了比较完整的闭环：输入可以解析为图，图可以进行环 判断和分层，无环图可以枚举或统计拓扑序，异常输入和环图也能被测试明确 覆盖。

4. 遇到的困难与解决过程

第一个困难是标准图拓扑序数量的确认。项目初期契约中把标准五节点图 `<A,C> <A,E> <B,C> <C,D>` 的合法拓扑序数量写成了 6 条，但在实现枚举器 并逐条检查后，发现该偏序实际有 7 条合法线性扩展。这个问题如果只看手算 很容易漏掉某个分支。我的处理方式是先让枚举算法保持确定性输出，再用测试 数据固定完整结果清单，最后把修正结论写入 [evidence/decisions/2026-09-18-标准图拓扑序数量修正.md](#)，避免后续模块继续 沿用错误期望。

第二个困难是“全部拓扑序枚举”和“性能”之间的矛盾。全部枚举本身可能出现 阶乘级结果，不能简单承诺任意大图都快速输出全部序列。因此我在设计中区分 了枚举模式和仅计数模式：枚举模式返回具体顺序，适合小图和演示；仅计数模式 不保存完整结果，适合验证较大但分支少的图。对于 30 节点和 60 节点稀疏链， 我把性能结果追加到 [evidence/benchmarks.csv](#)，同时在详细设计中说明这类 数据验证的是固定开销，不代表高分支图最坏情况。

第三个困难是纯逻辑层和 GUI 依赖的边界。项目要求 `app.models` 不能引入 Qt，但测试环境中 PySide6 插件加载可能影响 CI 判断。修复时我没有把 GUI 逻辑放进 models，也没有为了通过测试放宽边界，而是调整 CI 隔离方式，确保 models 层仍然可以 作为无 Qt 依赖的算法模块被独立导入和测试。

第四个困难是错误输入的口径统一。输入既要对用户友好，又不能让错误被静默 吞掉。最后采用的做法是：空白行跳过，常见全角符号和 `A,B` 形式宽容接受； 但缺少节点名、格式多余、自环等问题必须返回明确错误，并保留原始行号。这 样 UI 层后续可以把问题定位到具体输入行，测试也可以直接断言错误位置。

5. 个人感想

这次项目让我更清楚地体会到，算法作业里的“实现拓扑排序”如果放进一个可交付 软件中，并不只是写出一段能跑的 Kahn 算法。真正需要处理的是边界、接口和 可复现性：输入格式要稳定，错误要能解释，结果顺序要确定，性能限制要讲清楚， 测试数据和决策也要能回头查到。

我负责的 models 层处在项目底部，它本身没有界面效果，但会直接影响事件播放、 图形布局、结果导出和测试报告。这个位置让我意识到基础模块的接口越清楚， 后续联调成本就越低。例如 `ParseError` 带行号、`CycleError` 单独公开、`Graph` 使用不可变数据结构、枚举输出保持字典序，这些细节在写的时候并不 复杂，但能减少其他模块对内部实现的猜测。

另一个收获是对算法复杂度的表达更谨慎。全序枚举天然可能爆炸，如果只把“极速模式”写成口号，很容易误导使用者和测试者。把“只计数、少存储”和“最坏情况下仍然要遍历全部线性扩展”区分开，是这次我认为比较重要的设计 经验。

最后，我也感受到团队项目中留档的重要性。标准图结果数量修正、异常契约、 测试数据和性能 CSV 都不是额外负担，而是在出现争议或回归时能快速定位问题 的依据。相比单人作业只追求最后能运行，这次项目更强调每个结论都能被代码、 测试和文档互相证明，这对我后续参与协作开发有直接帮助。